

7 Tarefas Multimodais

Julia + Pluto, Scratch e material concreto — cobrindo os 12 temas centrais do curso, agrupados em 7 tarefas

Esta versão substitui os dois conjuntos anteriores (7 tarefas dos blocos 2-8 + 5 tarefas do módulo complementar) por um único conjunto de 7 tarefas, agrupando temas relacionados: Indução+Dedução; Problemas e Algoritmos; Representação+Arrays; Decomposição+Generalização+Transformação; Abstração e Especialização; Definindo Tipos+Ações por Categoria; e Complexidade de Algoritmos. Cada tarefa continua em três versões — Julia + Pluto, Scratch e material concreto — com o mesmo critério qualitativo do restante do curso.

TAREFA 1

Indução e Dedução — da hipótese à regra aplicada

Aula 2-3 • Pensamento Indutivo e Dedutivo • Pensamento Computacional

OBJETIVO DA TAREFA

Formular uma hipótese a partir de casos observados (indução) e, em seguida, aplicar uma regra geral já definida a casos específicos (dedução) — os dois raciocínios que se completam, trabalhados nas Aulas 2 e 3.

VERSÃO EM JULIA + PLUTO

Parte 1 — Indução: observem a sequência e proponham uma regra para o próximo termo.

```
sequencia = [3, 6, 9, 12]
proximo_termo(seq) = seq[end] + 3    # hipótese: soma 3 ao anterior

# Parte 2 – Dedução: apliquem uma regra geral já definida a casos específicos
multiplo_de_3(n) = n % 3 == 0 ? "múltiplo de 3" : "não é múltiplo de 3"
casos = [9, 10, 15, 20]
resultados = multiplo_de_3.(casos)
```

Na Parte 1, testem a hipótese com novos termos e um contraexemplo. Na Parte 2, apliquem a regra a pelo menos 4 casos, incluindo um caso-limite (um múltiplo exato de 3), e discutam a diferença entre propor uma regra (indução) e aplicá-la (dedução).

VERSÃO EM SCRATCH

- Parte indução: criem a lista "sequência" (3, 6, 9, 12), peçam a regra com "pergunte e espere" e testem com um bloco personalizado.
- Parte dedução: criem um bloco "é múltiplo de 3?" usando "se ... então ... senão" com o operador de resto da divisão, e apliquem a uma lista de números.
- Comparem as duas partes: numa, a turma descobre a regra; na outra, a regra já existe e só precisa ser aplicada.

VERSÃO COM MATERIAL CONCRETO

- Parte indução: cartões com sequência de bolinhas (1,2,3, depois 4,6,8) e um "cartão-armadilha" (contraexemplo).
- Parte dedução: fluxograma em cartolina ("é múltiplo de 3?") aplicado a fichas de números diferentes, incluindo um múltiplo exato.
- Discutam em roda: em qual das duas partes a turma teve que criar a regra, e em qual só seguiu uma regra já pronta?

Critério qualitativo recorrente: *investigação e revisão de hipóteses (indução) e aplicação de regras gerais a casos específicos (dedução).*

TAREFA 2

Problemas e Algoritmos — entrada, processamento e saída

Aula 4 • Problemas e Algoritmos • Pensamento Computacional

OBJETIVO DA TAREFA

Separar e identificar as três fases de um algoritmo (entrada, processamento, saída) num problema com decisão — cálculo de média com situação de aprovação — testando casos diversos e um caso-limite, como praticado na Aula 4.

VERSÃO EM JULIA + PLUTO

Separem claramente as três fases no código: dados de entrada, cálculo e resultado de saída.

```
notas = [7.5, 6.0, 8.2]           # entrada
media = sum(notas) / length(notas) # processamento
situacao = media >= 6.0 ? "aprovado" : "em recuperação" # saída
```

Testem com pelo menos 3 conjuntos de notas diferentes, incluindo um caso-limite (média exatamente 6.0), e apontem no código onde está cada uma das três fases.

VERSÃO EM SCRATCH

- Usem 3 blocos "pergunte e espere" para pedir 3 notas.
- Calculem "soma" e "média" com variáveis e blocos de operadores.
- Usem "se ... então ... senão" para decidir "aprovado" / "em recuperação", mostrando o resultado com "diga".
- Marquem com comentários ou post-its ao lado do script qual bloco representa entrada, qual representa processamento e qual representa saída.

VERSÃO COM MATERIAL CONCRETO

- Montem um "algoritmo humano": uma pessoa representa a entrada (segura fichas com as notas), outra representa o processamento (faz a conta numa lousa) e uma terceira representa a saída (anuncia o resultado).
- Testem com 3 conjuntos de notas diferentes, escritos em fichas, passando-as fisicamente entre as três estações.
- O professor entrega um conjunto de notas com um valor inválido (ex.: uma nota "25") e o grupo discute em qual estação esse erro deveria ser detectado.

Critério qualitativo recorrente: decomposição do algoritmo em fases e teste/depuração.

TAREFA 3

Representação e Arrays — organizando coleções de dados

Aula 5, 13 · Representação de Processos e Dados / Arrays e Estruturas de Dados · Pensamento Computacional

OBJETIVO DA TAREFA

Praticar transnumeração — a mesma informação em formas diferentes — e depois organizar dados em um array aninhado, percorrendo a estrutura para calcular um resultado, unindo o trabalho das Aulas 5 e 13.

VERSÃO EM JULIA + PLUTO

Primeiro representem os mesmos dados de formas diferentes; depois organizem dados em um array aninhado.

```
frutas = ["maçã", "banana", "maçã", "uva", "banana", "maçã"]

contagem = Dict{String, Int}()
for f in frutas
    contagem[f] = get(contagem, f, 0) + 1
end

# Array aninhado: notas de 2 alunos
notas = [
    [8.5, 9.0, 7.5], # Ana
    [7.0, 6.5, 8.0], # João
]
media_ana = sum(notas[1]) / length(notas[1])
```

Desenhem um gráfico de barras em texto a partir de 'contagem' e comparem com a lista original. Depois, percorram o array aninhado 'notas' com um laço for e calculem a média de cada aluno.

VERSÃO EM SCRATCH

- Criem uma lista "frutas" com itens repetidos e contem as ocorrências em variáveis (simulando um dicionário); desenhem barras proporcionais com o bloco de caneta.
- Criem uma lista "alunos" (array 1D) e, para cada aluno, uma lista separada de notas (representando o array aninhado).
- Somem os itens de cada lista de notas e dividam pela quantidade, mostrando a média de cada aluno com "diga".

VERSÃO COM MATERIAL CONCRETO

- Espalhem fichas de frutas repetidas sobre a mesa (lista), depois reorganizem em pilhas por tipo (contagem) e depois em colunas lado a lado (gráfico) — comparando as três representações.
- Organizem uma fileira de fichas com nomes de alunos e, abaixo de cada nome, um envelope com fichas de notas (array aninhado); cada grupo calcula a média do seu envelope.
- Discutam: qual representação de frutas facilita ver "qual aparece mais"? E como vocês localizariam "a segunda nota do primeiro aluno" dentro dos envelopes?

Critério qualitativo recorrente: representação, manipulação e transnumeração; organização de coleções, incluindo estruturas aninhadas.

TAREFA 4

Decomposição, Generalização e Transformação — as três técnicas juntas

Aula 6, 7, 8 · Decomposição, Generalização e Transformação · Pensamento Computacional

OBJETIVO DA TAREFA

Aplicar as três técnicas de construção de algoritmos num só problema: decompor o cálculo do preço de uma compra em subfunções, generalizá-las com parâmetros e, por fim, transformá-las num pipeline aplicado a vários pedidos — unindo as Aulas 6, 7 e 8.

VERSÃO EM JULIA + PLUTO

Decomponham o cálculo, generalizem com parâmetros, e apliquem o pipeline a uma lista de pedidos.

```
# Decomposição: subfunções menores
subtotal(precos) = sum(precos)
frete(peso) = peso > 5 ? 20.0 : 10.0
desconto(valor, cupom) = cupom ? valor * 0.9 : valor

# Generalização: uma função parametrizada que combina as partes
preco_final(precos, peso, cupom) = desconto(subtotal(precos), cupom) +
frete(peso)

# Transformação: pipeline aplicado a uma lista de pedidos
pedidos = [
    (precos=[10.0, 20.0], peso=3, cupom=false),
    (precos=[50.0, 5.0], peso=6, cupom=true),
]
totais = [preco_final(p.precos, p.peso, p.cupom) for p in pedidos]
```

Testem cada subfunção isoladamente (decomposição); testem `preco_final` com um caso extremo, como `peso = 0` (generalização); e comparem, para cada pedido, o dado bruto (entrada) com o total calculado (saída) — o antes e o depois do pipeline (transformação).

VERSÃO EM SCRATCH

- Criem 3 blocos personalizados: "calcular subtotal", "calcular frete" e "aplicar desconto" — testando cada um isoladamente (decomposição).
- Combinem os três num bloco "preço final" reutilizável, com parâmetros para qualquer pedido (generalização), e testem um caso extremo (peso 0).
- Rodem o bloco combinado para uma lista de 2 a 3 "pedidos" diferentes, mostrando o total de cada um e comparando com o pedido bruto (transformação em pipeline).

VERSÃO COM MATERIAL CONCRETO

- 3 grupos calculam cada parte separadamente em cartazes: "subtotal", "frete", "desconto" (decomposição).
- Um quarto grupo ("montador") combina as três partes numa fórmula geral, testada com pedidos variados, incluindo um caso extremo como peso 0 (generalização).
- Montem uma linha de montagem física: uma fila de fichas de pedidos passa pelas três estações em sequência até chegar ao preço final, e comparem o pedido bruto com o resultado final de cada ficha (transformação).

Critério qualitativo recorrente: *decomposição, generalização e transformação como técnicas de construção de algoritmos.*

TAREFA 5

Abstração e Especialização — do conceito geral ao específico

Aula 10 · Abstração e Especialização em Computação · Pensamento Computacional

OBJETIVO DA TAREFA

Criar um tipo genérico (abstrato) e pelo menos duas especializações concretas que herdam o que é comum e acrescentam o que é próprio — a mesma dupla abstração/especialização trabalhada na Aula 10.

VERSÃO EM JULIA + PLUTO

Definam um tipo abstrato e duas especializações com um comportamento comum (descrever).

```
abstract type Veiculo end

struct Carro <: Veiculo
    rodas::Int
    combustivel::String
end

struct Bicicleta <: Veiculo
    rodas::Int
end

descrever(v::Carro) = "Carro com $(v.rodas) rodas, movido a $(v.combustivel)"
descrever(v::Bicicleta) = "Bicicleta com $(v.rodas) rodas"

veiculos = [Carro(4, "gasolina"), Bicicleta(2)]
```

Testem a função *descrever* para cada veículo da lista e acrescentem um terceiro tipo (ex.: *Moto*) especializando *Veiculo* com pelo menos um atributo próprio.

VERSÃO EM SCRATCH

- Criem dois sprites especializados, "Carro" e "Bicicleta" (o "Veículo" genérico existe só como ideia, sem sprite próprio).
- Deem a cada sprite variáveis próprias (ex.: "rodas" nos dois, "combustível" só no Carro).
- Criem em cada sprite um bloco personalizado "descrever" que mostra suas características com "diga".
- Executem clicando em cada sprite e comparem as duas descrições.

VERSÃO COM MATERIAL CONCRETO

- Façam um cartaz central "Veículo" (conceito genérico) com setas apontando para dois cartazes "filhos": "Carro" e "Bicicleta".
- Em cada cartaz-filho, listem os atributos próprios (Carro: 4 rodas, combustível; Bicicleta: 2 rodas) — e uma linha comum "herdada" do cartaz genérico (todo veículo tem rodas).
- Discutam: se quiséssemos acrescentar um terceiro veículo, como "Patinete", o que ele herdaria do cartaz genérico, e o que seria só dele?

Critério qualitativo recorrente: *abstração (destacar o essencial) e especialização (hierarquia do genérico ao específico).*

TAREFA 6

Tipos Próprios e Ações por Categoria

Aula 11-12 • Definindo Novos Tipos e Ações Ligadas a Categorias • Pensamento Computacional

OBJETIVO DA TAREFA

Criar tipos próprios (um fixo, um mutável) e, em seguida, definir uma ação diferente para cada categoria de tipo — unindo a criação de tipos (Aula 11) com o reconhecimento de que a categoria define a ação certa (Aula 12).

VERSÃO EM JULIA + PLUTO

Criem um tipo fixo e um mutável, e uma função processar com um comportamento específico para cada um.

```
struct Aluno
    nome::String
    matricula::Int
    notas::Vector{Float64}
end

mutable struct Estoque
    produto::String
    quantidade::Int
end

processar(a::Aluno) = "Média de $(a.nome): $(sum(a.notas)/length(a.notas))"
processar(e::Estoque) = "Estoque de $(e.produto): $(e.quantidade) unidades"

itens = [Aluno("Maria", 1, [8.0, 9.0]), Estoque("Caderno", 50)]
for item in itens
    println(processar(item))
end
```

Testem processar para os dois itens da lista e expliquem por que a ação de calcular média não faria sentido aplicada ao Estoque, e vice-versa.

VERSÃO EM SCRATCH

- Criem um sprite "Aluno" (dados fixos: nome, matrícula, notas) e um sprite "Estoque" (quantidade alterável pelo bloco "mude quantidade por").
- Deem a cada sprite um bloco "processar" próprio: o do Aluno calcula a média das notas, o do Estoque mostra a quantidade atual.
- Executem clicando em cada sprite e comparem as duas ações.

VERSÃO COM MATERIAL CONCRETO

- Preençam uma "Ficha de Aluno" a caneta (não pode ser apagada — tipo fixo) e uma "Ficha de Estoque" a lápis (quantidade apagável e reescrevível — tipo mutável).
- Ao lado de cada ficha, escrevam a ação própria daquela categoria (aluno: calcular média das notas; estoque: contar unidades) e peçam que o grupo execute cada ação manualmente.
- Discutam: por que calcular média faz sentido para o aluno mas não para o estoque, e por que atualizar quantidade faz sentido para o estoque mas não para o aluno?

Critério qualitativo recorrente: criação de tipos próprios, mutabilidade, e reconhecimento de que a categoria define a ação apropriada.

TAREFA 7

Complexidade de Algoritmos — contando passos, não segundos

Aula 14 · Complexidade de Algoritmos · Pensamento Computacional

OBJETIVO DA TAREFA

Construir a função de custo $T(n)$ a partir da contagem de passos, comparando o crescimento de um laço simples (linear) com o de um laço duplo (quadrático) — como praticado na Aula 14.

VERSÃO EM JULIA + PLUTO

Contem os passos de duas funções que processam a mesma lista de tamanhos diferentes.

```
function soma_simples(v)
    s = 0
    for x in v
        s += x          # 1 passo por elemento ->  $T(n) = n$ 
    end
    return s
end

function pares_iguais(v)
    contador = 0
    for i in 1:length(v)
        for j in 1:length(v)
            if v[i] == v[j]
                contador += 1    # 1 passo por par ->  $T(n) = n^2$ 
            end
        end
    end
    return contador
end
```

Contem manualmente quantas vezes a linha marcada roda para $n = 4$, depois testem com $n = 8$: a `soma_simples` dobrou de trabalho, ou a `pares_iguais` quadruplicou? Registrem os dois valores.

VERSÃO EM SCRATCH

- Montem um script com um laço "repita (tamanho da lista) vezes" que soma os itens de uma lista, usando uma variável "contador de passos" que aumenta 1 a cada repetição.
- Montem um segundo script com dois laços aninhados ("repita" dentro de "repita") sobre a mesma lista, também contando os passos.
- Testem os dois scripts com listas de tamanho 4 e depois 8, anotando o valor final do contador de passos em cada caso, e comparem o crescimento.

VERSÃO COM MATERIAL CONCRETO

- Organizem uma fileira de 4 fichas numeradas (representando os elementos de um vetor).
- Atividade linear: um aluno toca em cada ficha uma única vez, em fileira, contando os toques em voz alta ($T(n) = n$).
- Atividade quadrática: para cada ficha tocada, o aluno percorre a fileira inteira de novo tocando todas as outras fichas (simulando o laço duplo), contando o total de toques ($T(n) = n^2$).

- Repitam as duas atividades com 8 fichas e comparem: quantas vezes o total de toques aumentou em cada atividade quando dobramos a quantidade de fichas?

Critério qualitativo recorrente: *construir a função de custo $T(n)$ e diferenciar crescimento linear de quadrático.*